
04 Task Performance 1 - Inventory System (C++ STL Vectors & Iterators)

Student Name: EDGARDO B. DEL ROSARIO JR.

Year and Section: BSIT-1C

Course Learning Objectives:

C01: Comprehend and clarify the characteristics, functions, and applications of essential data structures (such as arrays, linked lists, stacks, queues, trees, and graphs).

C02: Create and execute data structures and algorithms in a programming language.

C04: Utilize suitable data structures and algorithms to address computational challenges.

Activity Tasks:

- Utilize the industry-standard **std::vector** to manage a resizable collection of items.

Submission Contents/ Requirements:

1. You must include the actual source code file and/or screenshots of the running program showing its successful operation.
2. Please ensure that your written answers or entries are provided on new, separate sheets.

Your PDF must be named as:

Class_LastName_FirstName_TP01.pdf (Ex: IT1a_DelaCruz_Juana_TP01.pdf)

Failure to follow instructions (wrong format, missing screenshot, no code inside PDF) will result in score deduction.

Scenario

You are building an inventory system for an RPG game. The player can pick up items (add to inventory) and drop items (remove from inventory). Since the inventory size changes frequently, a static array is unsuitable.

Requirements

1. Include `<vector>` and `<algorithm>`.
2. Create a `vector<string>` named `inventory`.
3. Implement a menu with the following options:
 - ☐ **1. Add Item:** Ask user for item name and `push_back`.
 - ☐ **2. Remove Item:** Ask user for item name. Use an **iterator** to search and `erase` the item. Hint: Use a loop or `std::find`.
 - ☐ **3. View Inventory:** Display all items and the total count (`size()`).
4. Ensure the program handles removing items that don't exist gracefully.

Sample Input/Output

```
1. Add Item
2. Remove Item
3. View Inventory
> 1
Enter item: Sword
> 1
Enter item: Potion
> 3
Inventory (2 items): Sword, Potion
> 2
Enter item to remove: Sword
Removed Sword.
> 3
Inventory (1 items): Potion
```

Criteria	Description	Points
STL Setup	Correct inclusion of <code><vector></code> and declaration of <code>vector<string></code> without syntax errors.	10 pts
Insertion Logic	Correct implementation of adding items to the inventory using <code>.push_back()</code> .	15 pts
Deletion Logic	Correct usage of iterators and <code>.erase()</code> to find and remove elements safely (handling both found/not-found cases).	25 pts
Total		50 pts

MY PROGRAM

SOURCE CODE(TEXT):

```
/*
Developed by: DevEJ_Del_Rosario
Fullname: Edgardo B. Del Rosario Jr.
Class: BSIT-1C
=====
Project: 04 Task Performance 1 - Inventory System (C++ STL Vectors & Iterators)
Developed by: DevEJ_Del_Rosario
Copyright (c) 2026. All Rights Reserved.
=====
This code is the original work of DevEJ_Del_Rosario

Users may utilize this code as a guide or for educational purposes.
However, claiming this code as one's own or using it for commercial purposes
without proper citation or acknowledgment of the original creator is strictly prohibited.
=====
*/
#include <fstream>

#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <chrono>
#include <iomanip>
#include <algorithm>
#include <limits>

using std::string;
//For Cleaning Before Running
//Note: Only Works on Offline(Windows)
void clearBeforeRunning(){
    std::cout << "\033[2J\033[1;1H";
}

// To add a delay
void delay(int time){
    std::this_thread::sleep_for(std::chrono::milliseconds(time));
}
```

```
// Optional fo Line Breaking
void Br() { std::cout << "\n"; }
```

```
////////////////////////////////////
//Vector Array of Namelist
std::vector<string> Inventory;
////////////////////////////////////
```

```
string addItem(){
// std::cin.ignore();
    string item;
    std::cout << "Enter item: "<<std::endl;
    std::getline(std::cin, item);

    return item;
    std::cin.ignore();
}
```

```
void viewInventory( ){
    int n = Inventory.size();
    // int itemNum = 1;

    std::cout << "Inventory:\n"
        << std::endl;
    for(int i = 0; i < n; i++)
    {

        std::cout <<i+1<< ". " << Inventory[i] << std::endl;
        // itemNum++;
        ;

    }
}
```

```
void removeItem(string Remove ){

int _remove;
bool confirmationLoop;

    auto target = find(Inventory.begin(), Inventory.end(), Remove);
    if(target != Inventory.end()){
        std::cout<<"Item found"<<std::endl;
        // _remove = target - box.begin();

        _remove = distance(Inventory.begin(), target);
        confirmationLoop = true;
```

```

    }

    else{
        std::cout<<"Item not found"<<std::endl;
        confirmationLoop = false;

    }
    char confirmation;

    while (confirmationLoop == true)
    {
        std::cout << "\nConfirm? "
            << "\"Y\" for yes and \"N\" for No\n"
            << std::endl;
        std::cin >> confirmation ;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        if (confirmation == 'Y' || confirmation == 'y')
        {

            string removedItem = Inventory[_remove];
            Inventory.erase(Inventory.begin() + _remove);
            std::cout<<"Item Removed: "<<removedItem<<std::endl;
            confirmationLoop = false;
        }
        else if (confirmation == 'N' || confirmation == 'n')
        {
            std::cout<<"Task Canceled"<<std::endl;
            confirmationLoop = false;
        }
        else
        {
            std::cout << "\n"
                << "invalid response" << "\n";
            confirmationLoop = true;
        }

        // return box;

    }
    // removeItem(Remove, Inventory);

    // drawStarts=false;

    //selectionLoop = true;
}

```

```

void ClearInventory() {

    char confirmation;
    bool confirmationLoop = true;

    while (confirmationLoop == true)
    {
        std::cout << "\nConfirm? "
            << "\"Y\" for yes and \"N\" for No\n"
            << std::endl;
        std::cin >> confirmation ;
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        if (confirmation == 'Y' || confirmation == 'y')
        {
            std::cout << "Deleting Items..." << std::endl;
            int k = Inventory.size();
            for(int i = 0; i < k; i++) {
                delay(500);
                Inventory.pop_back();
            }
            clearBeforeRunning();
            std::cout << "\"INVENTORY CLEARED!\"" << std::endl;

            confirmationLoop = false;
        }
        else if (confirmation == 'N' || confirmation == 'n')
        {
            std::cout<<"Task Canceled"<<std::endl;
            confirmationLoop = false;
        }
        else
        {
            std::cout << "\n"
                << "invalid response" << "\n";
            confirmationLoop = true;
        }
    }

}

```

```

void loadInventory() {
    std::ifstream file("inventory.txt");
    std::string item;
    while (std::getline(file, item))
        Inventory.push_back(item);
}

```

```

void saveInventory() {
    std::ofstream file("inventory.txt");
    for (auto& item : Inventory)
        file << item << "\n";
}

```

```

int main(){

```

```

    clearBeforeRunning();
    loadInventory();

```

```

// bool drawStarts = false;
string selection;

```

```

bool selectionLoop = true;
while (selectionLoop == true)
{

```

```

    std::cout << "\n"
        << "Numbers of items stored: " << Inventory.size()
        << "\n"
        << "Enter (1) To Add Item.\n"
        << "Enter (2) To Remove A Item\n"
        << "Enter (3) To View The Inventory.\n"
        << "Enter (4) To Clear The Inventory.\n"
        << "Save & Quit (X) \n"
        << std::endl;
    std::getline(std::cin, selection);

```

```

    // std::cin >> selection;
    // std::cin.getline(selection, 1);

```

```

if (selection == "1" )
{
    Inventory.push_back(addItem());
}

```



```

else if ( selection == "2" ) {
    //
    if(Inventory.empty()){
        std::cout<<"<!EMPTY INVETORY!>"<<std::endl;
        selectionLoop = true;
    }
    else{
        string Remove;

        // std::string Confirmation;
        std::cout<<"Enter item to be Removed: ";
        std::getline(std::cin, Remove);

removeItem( Remove );

    }

}

    else if (selection == "3" )
{
    if(Inventory.size() != 0){
        viewInventory( );
    }
    else{
        std::cout<<"<!EMPTY INVETORY!>"<<std::endl;
    }
    // int size =Names.l
    // while (Names.size() > 0){

    //}
}
else if (selection == "4" )
{
    if(!Inventory.empty()){
        ClearInventory( );
}

    else if (selection == "X" || selection == "x" )
{
    selectionLoop = !true;
}

    else{
        std::cout<<"<!THE INVETORY IS ALREADY EMPTY!>"<<std::endl;
    }
}
}

```

```
else
{
std::cout << "\n\n\n"
<< "Invalid! Try Again" << "\n\n"
<< std::endl;

selectionLoop = true;
}
saveInventory();
}
return 0;
}
```

MY CODE SNIPPET VIA GITHUB:

[https://github.com/DevEJDelRosario26/SCHOOL-PROJECTS/blob/main/DSA/LE04/DevEJD_InventorySystem\(C%2B%2B_STL_Vectors_%26_Iterators\).cpp](https://github.com/DevEJDelRosario26/SCHOOL-PROJECTS/blob/main/DSA/LE04/DevEJD_InventorySystem(C%2B%2B_STL_Vectors_%26_Iterators).cpp)

PROGRAM SAMPLE OUTPUT::

```
TERMINAL  CHAT  ... | [ ] X
cppdbg: DevEJD_InventorySystem(C++ STL Vectors_& Iterators).exe + [ ] [ ] ...

PS C:\PROGRAMMING_AND_DEVELOPMENT\CODES> & 'c:\Users\EJ Del Rosario\.vscode\extensions\ms-vscode.cpptools-1.30.5-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-vld0u22c.lnt' '--stdout=Microsoft-MIEngine-Out-roek0a2.pf2' '--stderr=Microsoft-MIEngine-Error-2rddy01q.51t' '--pid=Microsoft-MIEngine-Pid-1ib1pvct.d41' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'

Numbers of items stored: 0
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)

1
Enter item:
Diamond Sword

Numbers of items stored: 1
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)

1
Enter item:
Diamon Pickaxe

Numbers of items stored: 2
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)

1
```

```
TERMINAL  cppdbg: DevEJD_InventorySystem(C++ STL Vectors_& Iterators).exe + [ ] [ ] ...

Numbers of items stored: 2
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)

1
Enter item:
Poisonus Potato

Numbers of items stored: 3
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)

3
Inventory:

1. Diamond Sword
2. Diamon Pickaxe
3. Poisonus Potato

Numbers of items stored: 3
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)

2
Enter item to be Removed: Poisonus Potato
Item found

Confirm? "Y" for yes and "N" for No

y
Item Removed: Poisonus Potato

Numbers of items stored: 2
Enter (1) To Add Item.
Enter (2) To Remove A Item
Enter (3) To View The Inventory.
Enter (4) To Clear The Inventory.
Save & Quit (X)
```